Plano de Respostas aos Riscos — Tennis Analytics (Entrega 2)

Oct 29, 2026 · @Daniel

Projeto: Tennis Analytics (Smart Tennis Arena) — D2LV

Data: 29/10/2026

Versão: 2.0 (Entrega 2; continua o Plano de Gerenciamento de Riscos da Entrega 1, de 12/09/2026)

Equipe: Daniel Kyoshi Moribe (Backend / Segurança), Leonardo Santos da Silva (Embarcado / IoT), Lucas de Lima Gutierrez (App Mobile — Fluxo da Sessão) e Vitor Kenzo Kanashiro (App Mobile — Análise, Web e Gerente do Projeto)

Código: `github.com/2026-2-NCC6/Projeto3`, branch `main`

1. Introdução

Dos 24 riscos de segurança tratados aqui, 21 estão Concluídos e 3 Em Andamento. Os 3 pendentes dependem de etapas fora do código: verificar o pareamento com criptografia na placa (S-02), testar o app num celular (S-05) e fazer o deploy (S-24). Todo trecho de código citado existe no repositório: os de antes da correção no commit `74ee92e`, os de depois na `main`. Cada correção aponta para o commit que a fez.

Os riscos S-01 a S-16 vêm da revisão de código que o grupo fez no repositório de 22/10/2026 (commit `74ee92e`), sobre o firmware e o aplicativo. Os riscos S-17 a S-24 são novos: surgiram com a API, o banco de dados e o site, que não existiam naquela data.

Componente	Tecnologia	Pasta
Firmware da raquete	Arduino UNO R4 WiFi, MPU-6050, ArduinoBLE	`SRC/arduino/tennis_imu`
Aplicativo	React Native com Expo SDK 54	`SRC/app`
API	NestJS, Prisma, PostgreSQL no Supabase	`SRC/backend`
Site de autoria e administração	React com Vite	`SRC/site`

No repositório. Os commits citados foram enviados ao GitHub em 29/10/2026 e mesclados na `main` (commit `e8a2fec`). Os workflows de testes e segurança passaram na `main` nessa data.

2. Tabela de Respostas aos Riscos

21 riscos estão Concluídos e 3 Em Andamento; a oportunidade OPP-01 também está Concluída. Concluído significa correção em commit na `main` mais teste automatizado ou verificação registrada (Anexo B). O nível residual proposto para cada risco, depois das respostas, está no Anexo E.

Como os campos foram preenchidos

- **Nível:** probabilidade × impacto, pelas réguas da seção 6 e pela matriz da seção 7 do Plano da Entrega 1. S-01 a S-16 e OPP-01 vêm da análise do grupo de 24/10/2026. S-17 a S-24 foram classificados agora, com as mesmas réguas, considerando o sistema sem os controles; S-17 e S-19 repetem a avaliação do R15 da Entrega 1 (base exposta por regra de acesso mal configurada).
- **Responsável:** para ações feitas, o autor do commit; para ações pendentes, o dono da trilha segundo os papéis da Entrega 1 (Leonardo: embarcado e IoT; Lucas: app; Daniel: backend e segurança).
- **Prazo:** para ações feitas, a data do commit ou da execução; para as pendentes, 06/11/2026, data da Entrega 2.

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-01	Comandos BLE aceitos de qualquer aparelho, sem autenticação	Alto (Média × Alto)	Mitigar	1. Desafio e resposta com HMAC-SHA256 e chave por raquete; só PING, CH: e PA: antes de autenticar; 3 provas erradas bloqueiam por 30 s (`290e028`). 2. Repetir os cenários em bancada com a placa.	1. Daniel Kyoshi Moribe. 2. Leonardo Santos da Silva	1. 25/10/2026. 2. 06/11/2026	Concluído

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-02	Dados do BLE trafegam em texto claro	Médio (Média × Baixo)	Mitigar + Aceitar (Ativo)	1. Nenhum segredo passa pelo canal (`290e028`). 2. Verificar pareamento com criptografia na UNO R4 WiFi. 3. Métricas de golpe legíveis aceitas no protótipo, por não terem dado pessoal.	1. Daniel Kyoshi Moribe. 2. Leonardo Santos da Silva	1. 25/10/2026. 2. 06/11/2026	Em Andamento
S-03	Aparelho falso se passando pela raquete	Baixo (Baixa × Médio)	Mitigar	1. A placa prova primeiro; o app recusa prova errada e firmware anterior a 1.1.0 (`290e028`). 2. Testar com um aparelho que anuncie o mesmo nome.	1. Daniel Kyoshi Moribe. 2. Leonardo Santos da Silva	1. 25/10/2026. 2. 06/11/2026	Concluído
S-04	Login sem autenticação e sem controle de acesso por perfil	Alto (Alta × Médio)	Mitigar	1. Login pela API, com uma conta só para app e site (`5f88e47` , `dadcbac`). 2. Perfis conferidos no servidor. 3. Refresh token no SecureStore do aparelho.	Daniel Kyoshi Moribe	23/10/2026 e 25/10/2026	Concluído
S-05	Dependências do aplicativo com vulnerabilidades conhecidas	Alto (Média × Alto)	Mitigar + Aceitar (Ativo)	1. Expo SDK 54: de 53 para 13 vulnerabilidades, nenhuma crítica (`8431cad`). 2. Aceitas `node-forge` e `image-size` , sem correção e só em ferramentas de build. 3. Build novo e teste no celular.	1 e 2. Daniel Kyoshi Moribe. 3. Lucas de Lima Gutierrez	1 e 2. 25/10/2026. 3. 06/11/2026	Em Andamento

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-06	Nome e e-mail pessoal de integrante fixos no código	Médio (Alta × Baixo)	Prevenir (Evitar)	1. Dados da conta autenticada no lugar dos fixos (`38d8bf9` , `dadcbac`). 2. Reescrever o histórico do Git, se o grupo decidir.	1. Daniel Kyoshi Moribe. 2. Grupo	1. 25/10/2026. 2. Decisão do grupo	Concluído
S-07	Conformidade com a LGPD declarada sem implementação	Alto (Alta × Médio)	Mitigar	1. Exportação, exclusão de conta, consentimentos, retenção e política de privacidade (`5f88e47` , `dadcbac`). 2. Testes automatizados de exportação, exclusão e retenção (`48b5726`).	Daniel Kyoshi Moribe	1. 23/10/2026 e 25/10/2026. 2. 29/10/2026	Concluído
S-08	Sem registro de comandos, falhas e paradas	Médio (Alta × Baixo)	Mitigar	1. Trilha de auditoria no banco, que a API não altera nem apaga (`5f88e47`).	Daniel Kyoshi Moribe	23/10/2026	Concluído
S-09	Tela de depuração acessível no app de uso normal	Médio (Média × Baixo)	Prevenir (Evitar)	1. Tela só em build de desenvolvimento (`38d8bf9`). 2. DBG:1 exige autenticação (`290e028`).	Daniel Kyoshi Moribe	25/10/2026	Concluído
S-10	Pasta `.expo` versionada no repositório	Baixo (Alta × Muito Baixo)	Prevenir (Evitar)	1. `.expo/` no `.gitignore` e fora do controle de versão (`38d8bf9`).	Daniel Kyoshi Moribe	25/10/2026	Concluído

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-11	Credenciais podem vazar pelo repositório	Médio (Baixa × Muito Alto)	Prevenir (Evitar) + Transferir	1. Segredos fora do Git e gitleaks no CI (`38d8bf9`). 2. Segredos de produção no painel do Render (`5f88e47`). 3. Push e primeira execução do workflow no GitHub, sem vazamentos.	Daniel Kyoshi Moribe	1 e 2. 23/10/2026 e 25/10/2026. 3. 29/10/2026	Concluído
S-12	Mensagens malformadas ou maliciosas pelo BLE	Baixo (Baixa × Médio)	Mitigar	1. Limite de 20 bytes no firmware e validação de formato e faixas no app (`74ee92e`). 2. Desafio e prova só com 8 caracteres hexadecimais (`290e028`).	1. Leonardo Santos da Silva. 2. Daniel Kyoshi Moribe	1. 22/10/2026. 2. 25/10/2026	Concluído
S-13	Golpes duplicados inflando a pontuação	Baixo (Baixa × Médio)	Mitigar	1. Sequência de 1 a 9999 por golpe no app (`74ee92e`). 2. UUID por evento e gravação sem duplicidade na API (`5f88e47`).	1. Leonardo Santos da Silva. 2. Daniel Kyoshi Moribe	1. 22/10/2026. 2. 23/10/2026	Concluído
S-14	Permissões excessivas no aplicativo	Muito Baixo (Muito Baixa × Baixo)	Prevenir (Evitar)	1. BLE sem localização no Android 12 ou superior e sem execução em segundo plano (`85d53f6`).	Leonardo Santos da Silva	20/10/2026	Concluído
S-15	Parada segura ou queda de conexão deixando a placa em estado indefinido	Médio (Baixa × Alto)	Mitigar	1. STOP leva a PARADO e a queda para a coleta no firmware (`74ee92e`). 2. A API só aceita REINICIAR em parada segura e o app espera a confirmação da placa (`5f88e47` , `dadcbac`). 3. Teste em bancada.	1 e 3. Leonardo Santos da Silva. 2. Daniel Kyoshi Moribe	1. 22/10/2026. 2. 23/10/2026 e 25/10/2026. 3. 06/11/2026	Concluído

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-16	Atualização do firmware sem controle de versão	Médio (Média × Médio)	Mitigar + Aceitar (Ativo)	1. Versão 1.1.0 informada após autenticar; app e API recusam versão anterior (`290e028`). 2. Atualização só por cabo USB, aceita no protótipo.	Daniel Kyoshi Moribe	25/10/2026	Concluído
S-17	Acesso a dados de outra conta pela API	Alto (Média × Muito Alto)	Prevenir (Evitar)	1. Rotas fechadas por padrão; perfil e dono conferidos em cada recurso (`5f88e47`).	Daniel Kyoshi Moribe	23/10/2026	Concluído
S-18	Ataques ao login da API	Alto (Média × Alto)	Mitigar	1. Limite de tentativas, bcrypt, erro genérico e refresh rotativo com detecção de reuso (`5f88e47`). 2. Testes do limite de tentativas, da resposta igual para e-mail inexistente e de tokens forjados, sem assinatura e expirados (`48b5726`).	Daniel Kyoshi Moribe	1. 23/10/2026. 2. 29/10/2026	Concluído
S-19	Privilégio excessivo no banco de dados	Alto (Média × Muito Alto)	Prevenir (Evitar)	1. Papel da API só com leitura e escrita de dados, schema próprio, RLS e senhas trocadas (`5f88e47`).	Daniel Kyoshi Moribe	23/10/2026	Concluído
S-20	Vazamento das chaves das raquetes guardadas no servidor	Médio (Baixa × Alto)	Mitigar	1. Chaves cifradas com AES-256-GCM, entregues só ao dono, sem cache e com auditoria (`290e028`).	Daniel Kyoshi Moribe	25/10/2026	Concluído

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
S-21	Configuração HTTP insegura na API	Médio (Média × Médio)	Prevenir (Evitar)	1. Helmet, CORS por lista, validação estrita, erros sem detalhes e no-store (`5f88e47` , `1050afe`). 2. Corpo grande demais responde 413 e a API recusa segredos de exemplo (`48b5726`). 3. Testes de cabeçalhos, CORS, validação e erros (`48b5726`) e varredura local com OWASP ZAP, sem achado real. 4. Repetir o ZAP na API implantada (S-24).	Daniel Kyoshi Moribe	1. 23/10/2026 e 25/10/2026. 2 e 3. 29/10/2026. 4. 06/11/2026	Concluído
S-22	Roubo de sessão e clickjacking no site	Médio (Baixa × Alto)	Prevenir (Evitar)	1. CSP sem script de terceiros, token de acesso só em memória e `frame-ancestors 'none'` (`e202385`).	Daniel Kyoshi Moribe	25/10/2026	Concluído
S-23	Dependências da API e do site com vulnerabilidades conhecidas	Alto (Média × Alto)	Mitigar	1. Site sem vulnerabilidades conhecidas (`e202385`). 2. `multer` forçado para 2.5.0: `npm audit` de produção da API sem vulnerabilidades (`48b5726`).	Daniel Kyoshi Moribe	1. 25/10/2026. 2. 29/10/2026	Concluído
S-24	Tráfego sem TLS fora do ambiente local	Alto (Média × Alto)	Transferir	1. TLS da API e do site a cargo do Render. 2. Fazer o deploy e repetir a varredura com OWASP ZAP na API implantada.	Daniel Kyoshi Moribe	06/11/2026	Em Andamento

ID do Risco	Descrição do Risco	Nível do Risco (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas	Responsável pela Ação	Prazo para Implementação	Status
OPP-01	Testes automatizados como rede de proteção para as próximas correções	Médio	Explorar (Oportunidade)	1. Dois workflows rodam testes, tipos, compilação do firmware, gitleaks e `npm audit` a cada push (`38d8bf9` , `5f88e47` , `e202385`). 2. Primeira execução no GitHub, na `main` , com sucesso.	Daniel Kyoshi Moribe	1. 23/10/2026 e 25/10/2026. 2. 29/10/2026	Concluído

3. Detalhamento das Estratégias de Resposta

3.1. Estratégias para Ameaças (Riscos Negativos)

- **Prevenir (Evitar):** eliminar a causa da ameaça. Usada em S-06 (dado pessoal fora do código), S-09 (tela de depuração fora do app de uso normal), S-10 (pasta local fora do repositório), S-11 (segredos nunca versionados), S-14 (o app não pede permissões de que não precisa), S-17 (rotas fechadas por padrão), S-19 (papel do banco sem permissão além do necessário), S-21 (configuração segura por padrão) e S-22 (sem script de terceiros no site).
- **Transferir:** repassar a terceiro a gestão de parte do risco. Usada em S-11, com os segredos de produção guardados no painel do Render, e em S-24, com o certificado TLS fornecido pelo Render. O grupo continua responsável por configurar os dois.
- **Mitigar:** reduzir probabilidade ou impacto a um nível aceitável. Usada em S-01, S-02, S-03, S-04, S-05, S-07, S-08, S-12, S-13, S-15, S-16, S-18, S-20 e S-23: autenticação no BLE e na API, validação de mensagens, auditoria, cifra das chaves e atualização de dependências.
- **Aceitar:**
  - **Aceitação ativa** em três casos. S-02: as métricas de golpe continuam legíveis no BLE, porque não têm dado pessoal; a contingência é ligar o pareamento com criptografia se a placa oferecer. S-05: `node-forge` e `image-size` ficam sem correção por enquanto, porque só existem em ferramentas de build; a auditoria a cada push mostra quando houver correção. S-16: a atualização do firmware continua só por cabo USB, que exige acesso físico à placa.
  - **Aceitação passiva:** nenhum risco.

3.2. Estratégias para Oportunidades (Riscos Positivos)

- **Explorar:** usada em OPP-01. Os testes automatizados e os dois workflows do GitHub Actions viraram rede de proteção: um push que quebre um teste, exponha um segredo ou traga dependência vulnerável aparece para todo o grupo.
- **Melhorar (ou Realçar), Compartilhar e Aceitar:** nenhuma oportunidade recebeu essas estratégias.

4. Monitoramento e Revisão

Este plano será revisado semanalmente, no bloco fixo de riscos da reunião de alinhamento do grupo definido no Plano da Entrega 1, e em cada marco: Entrega 2 (06/11/2026) e apresentação na Semana FECAP de Tecnologia (17 a 24/11/2026). O campo Status é atualizado conforme o progresso das ações: Não Iniciado, Em Andamento, Concluído ou Cancelado. Um risco só passa a Concluído quando a correção está em commit e há teste ou verificação registrada.

Novos riscos entram no registro quando surgirem, e os existentes são reavaliados conforme o projeto avança. Entre uma reunião e outra, os workflows `backend.yml` e `seguranca.yml` funcionam como monitoramento contínuo a cada push. O nível residual proposto para cada risco (Anexo E) será confirmado pelo grupo na primeira reunião depois desta versão.

Aprovação

Papéis conforme o Plano de Gerenciamento de Riscos da Entrega 1.

Nome	Papel	Assinatura	Data
Daniel Kyoshi Moribe	Backend e Segurança		
Leonardo Santos da Silva	Embarcado e IoT		
Lucas de Lima Gutierrez	App Mobile — Fluxo da Sessão		
Vitor Kenzo Kanashiro	App Mobile — Análise, Web e Gerente de Projeto		

Metodologia. Revisão manual do código (análise estática) do firmware, do app, da API e do site; testes automatizados, inclusive testes que reproduzem ataques à API; auditoria de dependências com `npm audit`; busca de segredos no histórico com gitleaks; e

varredura dinâmica da API, rodando localmente, com o OWASP ZAP. Os riscos S-17, S-18, S-21 e S-23 citam a categoria correspondente do OWASP Top 10 (2021). O documento segue o modelo de Plano de Respostas aos Riscos da disciplina.

Anexo A — Detalhamento por risco

Para cada risco: o problema, a evidência no commit de 22/10/2026 (`74ee92e`), a correção com o commit e o arquivo atual, como foi verificada e o que ainda sobra. Os números de linha são os da branch `feat/seguranca` no commit `8431cad` , salvo indicação contrária.

S-01 — Comandos BLE aceitos de qualquer aparelho

Problema. A característica de comandos aceitava escrita de qualquer aparelho conectado, e o firmware executava START, STOP e DBG sem saber quem enviou. Alguém ao alcance do Bluetooth podia interromper uma sessão ou reiniciar a coleta depois de uma parada segura.

Antes (`74ee92e` , `tennis_imu.ino`): a função `processarComando(String cmd)` (linha 129) executa `START` (linha 134) e `STOP` (linha 146) sem nenhuma checagem de origem.

Correção (commit `290e028`). Desafio e resposta com HMAC-SHA256 e chave de 128 bits por raquete:

```
// autenticacao.h, linha 44: pelo BLE, só PING, CH: e PA: antes de autenticar
return strcmp(cmd, "PING") == 0 || strncmp(cmd, "CH:", 3) == 0 ||
strncmp(cmd, "PA:", 3) == 0;
// autenticacao.h, linha 64: cada desafio aceita uma única prova
temDesafio = false;  // uma prova por desafio, certa ou errada
// autenticacao.h, linhas 74 a 77: 3 provas erradas bloqueiam por 30 s
if (++tentativas >= MAX_TENTATIVAS) { ... bloqueadoAte = (uint32_t)agoraMs +
BLOQUEIO_MS;
// tennis_imu.ino, linhas 191 e 192
if (!autenticacao.comandoPermitido(cmd.c_str(), viaBle)) {
publicar("ERR:AUTH"); return; }
```

A chave fica em `segredo.h` , fora do Git; sem esse arquivo o firmware não compila (linhas 38 a 41). Cada raquete tem a sua chave, cadastrada na API pelo dono (S-20).

Verificação. 27 cenários do firmware no PC passam (Anexo B), entre eles "sem autenticação: START, STOP, DBG:1 e FB recusados pelo BLE", "prova capturada da conexão anterior é recusada (replay)" e "3 provas erradas derrubam a conexão". O firmware compila para a UNO R4 WiFi com o `arduino-cli` . Falta repetir os cenários com a placa.

Residual. A porta serial aceita comandos sem autenticação, porque exige acesso físico à placa. Os desafios não vêm de gerador criptográfico: na placa, SHA-256 sobre ruído analógico, tempo e contador (`tennis_imu.ino` , linhas 162 a 177); no app, SHA-256 sobre `Math.random()` , tempo e contador (`hmac.ts` , linha 97).

S-02 — Dados trafegam pelo BLE em texto claro

Problema. As mensagens são texto ASCII sem criptografia, como `S:F,6.9,18.2,2,14` (lado, pico de aceleração, pico de rotação, força e sequência). Um sniffer BLE próximo consegue lê-las.

Evidência. As características não pedem criptografia (`tennis_imu.ino` , linhas 104 e 105):

```
BLEStringCharacteristic dadosChar(CHAR_DADOS_UUID, BLERead | BLENotify,
MAX_MSG);
BLEStringCharacteristic cmdChar(CHAR_CMD_UUID, BLEWrite |
BLEWriteWithoutResponse, MAX_MSG);
```

Resposta. Com S-01, nenhum segredo passa pelo canal: trafegam só os dois desafios e provas de 4 bytes que valem para uma conexão. As métricas de golpe continuam legíveis e não identificam a pessoa. Falta verificar se a ArduinoBLE oferece pareamento com criptografia na UNO R4 WiFi.

Verificação. O cenário "não existe token em texto claro" passa no PC. Não houve captura com sniffer.

Residual. Métricas de golpe legíveis por quem estiver ao alcance do Bluetooth, aceitas de forma ativa no protótipo.

S-03 — Aparelho falso se passando pela raquete

Problema. O app procura a raquete pelo nome e pelo UUID do serviço, os dois publicados no repositório. Um aparelho que anunciasse esses valores podia enviar golpes falsos.

Correção (commit `290e028`). A placa prova primeiro que conhece a chave. O app só envia a própria prova se a da placa conferir (`SRC/app/src/ble/autenticacao.ts` , linha 38):

```
if (resposta.tipo !== 'desafio' || resposta.pb !== prova(chave, 'B', na,
resposta.nb)) throw new ErroAutenticacao('APARELHO_FALSO');
```

O app também recusa firmware anterior a 1.1.0 (linha 44).

Verificação. Os testes do app `hmac.test.ts` e `autenticacao.test.ts` e o firmware usam o mesmo vetor: chave `000102…0F` , Na `1A2B3C4D` , Nb `9F8E7D6C` , PB `BF55C708` e PA `83E700B5` . Falta testar com um aparelho real anunciando o mesmo nome.

Residual. Quem obtiver a chave de uma raquete consegue imitar aquela raquete.

S-04 — Login sem autenticação e sem perfis

Problema. Os botões Entrar e Criar conta só navegavam, e Esqueci minha senha não fazia nada.

Antes (`74ee92e` , `LoginScreen.tsx`):

```
47  <GhostButton label="Esqueci minha senha" onPress={() => {}} ... />
51  <PrimaryButton label="Entrar" onPress={() =>
navigation.replace('MainTabs')} />
52  <OutlineButton label="Criar conta" onPress={() =>
navigation.replace('MainTabs')} />
```

Correção (commits `5f88e47` na API e `dadcbac` no app). O login passou a ser feito pela própria API, com uma conta só para app e site. Os perfis jogador, autor e administrador são conferidos no servidor em cada rota (S-17). As telas internas só existem com usuário autenticado. O refresh token fica no SecureStore do aparelho (`AuthContext.tsx` , linha 62). As defesas da API estão em S-18. O botão Esqueci minha senha foi retirado, porque não há envio de e-mail.

Verificação. Testes E2E da API: "admin promove um cadastro a autor", "jogador não cria treino" e "reuso de refresh token revoga a família". O login pelo site foi testado no navegador em 25/10/2026.

Residual. Não há recuperação de senha nem confirmação de e-mail.

S-05 — Dependências do aplicativo com vulnerabilidades conhecidas

Problema. Em 22/10/2026, o `npm audit` do app apontava 53 vulnerabilidades: 2 críticas, 21 altas, 29 moderadas e 1 baixa, a maioria nas ferramentas do Expo SDK 51.

Correção (commit `8431cad`). App migrado para o Expo SDK 54 (React Native 0.81). Escolhemos o 54 e não o mais recente (58) porque a biblioteca de BLE é comprovada até o React Native 0.81. O `postcss` foi forçado para 8.5.31.

Verificação. `npm audit` em 28/10/2026: 13 vulnerabilidades, 2 altas e 11 baixas, nenhuma crítica. As 2 altas são `node-forge` , sem versão corrigida, e `image-size` 1.x, cuja correção (2.x) quebra o Metro. As 11 baixas vêm de um alerta novo do `deepmerge` , usado pelas ferramentas de configuração do Expo. Todas estão em ferramentas de build; nenhuma está no código que vai para o aparelho. Testes, tipos, `expo-doctor` e o pacote Android passam.

Residual. Falta gerar o build novo e testar o app no aparelho, principalmente o BLE. Se ele falhar, o plano B é desligar a New Architecture no `app.json` .

S-06 — Nome e e-mail pessoal de integrante no código

Problema. O repositório é público, e o Perfil mostrava o nome e o e-mail pessoal de um integrante.

Antes (`74ee92e`): `PerfilScreen.tsx` linha 47 (nome) e 48 (e-mail, omitido aqui); `InicioScreen.tsx` linha 57 (nome na saudação).

Correção (commits `38d8bf9` e `dadcbac`). As telas mostram os dados da conta autenticada, vindos da API.

Verificação. A busca pelo nome e pelo e-mail em `SRC/` não encontra nenhuma ocorrência.

Residual. O e-mail continua nos commits antigos. Retirá-lo exige reescrever o histórico do Git, decisão do grupo.

S-07 — Conformidade com a LGPD declarada sem implementação

Problema. O Perfil afirmava conformidade com a LGPD e cadastro só para maiores de 18 anos, sem cadastro nem verificação. O botão Exportar meus dados não fazia nada.

Antes (`74ee92e` , `PerfilScreen.tsx`): linha 126, "Dados de treino tratados conforme a LGPD. Cadastro restrito a maiores de 18 anos."; linha 131, `<GhostButton label="Exportar meus dados" onPress={() => {}} />` .

Correção (commits `5f88e47` e `dadcbac`):

- Exportação: `GET /me/exportar` devolve os dados da conta em JSON, compartilhados pelo app (`PerfilScreen.tsx` , linhas 108 e 109).
- Exclusão: `DELETE /me` com confirmação de senha; a conta é anonimizada para preservar a auditoria (`conta.service.ts` , linha 66).
- Cadastro: declaração de maioridade e aceite dos termos (`CadastroScreen.tsx` , linhas 108 e 111). Menores entram como dependentes, com consentimento do responsável.
- Política de privacidade: tela `PrivacidadeScreen.tsx` , aberta pelo Login e pelo Perfil.
- Telemetria: os golpes detalhados só vão à plataforma mãe com consentimento; para dependentes, também o do responsável (`sessoes.service.ts` , linha 266).
- Retenção: um job diário apaga os golpes depois de 180 dias (titular) ou 30 dias (dependente) e mantém o resumo (`retencao.service.ts` , linha 25).

Verificação (commit `48b5726` , `test/seguranca.e2e-spec.ts`). Três testes E2E: "exporta os dados da própria conta, sem senha e sem dados de outras contas"; "exclui a conta só com a senha certa; depois disso o login falha e os praticantes somem"; e "a retenção apaga golpes antigos, com prazo menor para dependentes, e mantém o resumo" (sessão de titular com 200 dias e de dependente com 40 dias perdem os golpes; a de titular com 40 dias e a de dependente com 5 dias mantêm).

Residual. A idade é declarada, não comprovada.

S-08 — Sem registro de comandos, falhas e paradas

Problema. Comandos, erros do sensor e paradas seguras não ficavam registrados. O RF23 pede esse registro, e o RF22 pede o motivo da parada.

Correção (commit `5f88e47`). A API grava uma trilha de auditoria no banco. O papel do banco usado pela API não tem permissão de alterar nem apagar esses registros (`REVOKE UPDATE, DELETE`, migration inicial, linha 359). A trilha tem 32 tipos de ação: autenticação, publicação, comandos de sessão com latência, paradas com motivo, entrega de chave de raquete e ações de administração. Comandos bloqueados e tentativas indevidas ficam com severidade INCIDENTE. O administrador consulta tudo no site, filtrando por incidente.

Verificação. Teste E2E "auditoria registra emergência e o comando bloqueado como incidente".

Residual. Nenhum relevante.

S-09 — Tela de depuração no app de uso normal

Problema. Um toque longo na sessão ou no pareamento abria a tela de depuração, que liga a telemetria bruta da placa e envia comandos fora do fluxo da sessão.

Antes (`74ee92e`): `RootNavigator.tsx` linha 81 registra a rota `Debug` em qualquer build; `SessaoScreen.tsx` linha 291 abre a tela com `onLongPress`.

Correção (commit `38d8bf9`). A rota e os atalhos só existem em build de desenvolvimento:

```
{__DEV__ && <Stack.Screen name="Debug" component={DebugScreen} />}
// RootNavigator.tsx, linha 98
onLongPress={__DEV__ ? () => navigation.navigate('Debug') : undefined}
// SessaoScreen.tsx, linha 471
```

No firmware, DBG:1 exige autenticação (commit `290e028`).

Verificação. Cenário do firmware "sem autenticação: START, STOP, DBG:1 e FB recusados pelo BLE" e verificação de tipos do app.

Residual. O código da tela continua no pacote, mas sem rota fora do modo de desenvolvimento.

S-10 — Pasta `.expo` versionada

Problema. `SRC/app/.expo/devices.json` estava no repositório com o identificador de instalação de um aparelho de desenvolvimento.

Correção (commit `38d8bf9`). `.expo/` entrou no `.gitignore` (linha 146) e a pasta saiu do controle de versão.

Verificação. Nenhum arquivo de `.expo/` está na branch `feat/seguranca`.

Residual. O arquivo continua nos commits anteriores.

S-11 — Credenciais vazando pelo repositório

Problema. O projeto passou a ter segredos de verdade: a chave de cada raquete, as senhas do banco e as chaves da API. Não havia verificação automática de segredos.

Correção (commits `38d8bf9`, `5f88e47` e `290e028`):

- `.env` e `segredo.h` ficam fora do Git (`.gitignore`, linha 149); o repositório só tem os modelos `.env.example` e `segredo.exemplo.h`.
- A API não sobe sem os segredos obrigatórios (`configuracao.ts`, linhas 23 a 34).
- No deploy, os segredos são gerados pelo Render ou preenchidos no painel, nunca no arquivo (`render.yaml`, `generateValue` e `sync: false`).
- O workflow `seguranca.yml` roda o gitleaks em todo o histórico a cada push.
- As senhas do banco foram trocadas depois de aparecerem numa conversa de trabalho; as antigas passaram a ser recusadas.

Verificação. gitleaks local em 28/10/2026: 27 commits analisados, nenhum vazamento.

Verificação no GitHub. Em 29/10/2026 a branch `feat/seguranca` foi enviada ao repositório e mesclada na `main` (commit `e8a2fec`). O workflow `seguranca.yml` rodou no GitHub Actions na `main` (execução 40812441260), e o job do gitleaks em todo o histórico terminou sem vazamentos.

S-12 — Mensagens BLE malformadas ou maliciosas

Problema. O canal BLE recebe texto livre. Uma mensagem malformada poderia corromper a pontuação ou estourar um buffer na placa.

Controles. Já existiam em `74ee92e` e foram ampliados no `290e028`:

```
const size_t MAX_MSG = 20;                                          //
tennis_imu.ino, linha 49
} else if (bufferSerial.length() < MAX_MSG) {                       //
tennis_imu.ino, linha 328
if (partes.length !== quantidade || !partes.every((p) => NUMERO.test(p)))
return null;   // protocolo.ts, linha 81
if (f < 1 || f > 3 || s > SEQ_MAX || Number(picoG) < 0 || Number(picoGiro) <
0) return null;   // protocolo.ts, linha 93
```

Desafio e prova só são aceitos com 8 caracteres hexadecimais, na placa e no app.

Verificação. Testes do app em `protocolo.test.ts`: "golpe truncado é descartado", "golpe com campo faltando ou inválido é descartado", "lixo é descartado", "resposta ao desafio (CR:) traz Nb e PB, cabe em 20 bytes e só aceita hex" e "versão do firmware (V:) só aceita x.y.z". No firmware: "desafio malformado (curto) é recusado" e "desafio malformado (não hex) é recusado".

S-13 — Golpes duplicados inflando a pontuação

Problema. Reenvios do BLE ou da rede poderiam contar o mesmo golpe duas vezes.

Controles. No app, cada golpe traz uma sequência de 1 a 9999, e `avaliarSeq` descarta duplicados e atrasados (`protocolo.ts`, linhas 156 a 159). Na API (commit `5f88e47`), cada evento tem um UUID gerado no app e a gravação ignora o que já existe (`sessoes.service.ts`, linha 300, `skipDuplicates`).

Verificação. Testes do app "seq: duplicado e atrasado são ignorados" e "seq: volta de 9999 para 1". Teste E2E "sincronização offline é idempotente" e o cenário da sessão, que reenvia um lote duplicado.

S-14 — Permissões excessivas no aplicativo

Problema. Apps de Bluetooth costumam pedir localização e execução em segundo plano.

Controles. Já em `74ee92e`: o plugin de BLE usa `"isBackgroundEnabled": false` e `"neverForLocation": true` (`app.json`, linhas 27 e 28). No Android 12 ou superior, o app pede só as permissões de Bluetooth; localização só nas versões anteriores, por exigência do sistema (`BleContext.tsx`, linhas 80 a 87).

Verificação. Em 25/10/2026, o `expo prebuild` do SDK 54 gerou o `AndroidManifest.xml` com `BLUETOOTH_SCAN` marcado como `neverForLocation`.

Residual. No Android 11 ou inferior, o sistema exige a permissão de localização para buscar aparelhos BLE.

S-15 — Parada segura ou queda de conexão deixando a placa em estado indefinido

Problema. Uma parada que não chega à placa, ou uma conexão que cai, poderia deixar a coleta rodando sem controle.

Controles. No firmware, STOP leva a PARADO (`tennis_imu.ino` , linha 228) e a queda da conexão volta a AGUARDANDO (linha 258). No app, a emergência vai primeiro à API e depois à placa, esperando a confirmação PARADO (`SessaoScreen.tsx` , linhas 261 e 262). Na API, em parada segura só REINICIAR é aceito (`estado.ts` , linha 24); qualquer outro comando responde 409 e é auditado como incidente (`sessoes.service.ts` , linhas 81 a 91), e REINICIAR exige confirmar que a área está segura (linhas 94 e 95).

Verificação. Cenário do firmware "queda da conexão desfaz a autenticação". Testes da máquina de estados "parada segura só aceita reiniciar". Teste E2E da sessão: emergência, comando recusado com 409, reinício e encerramento.

Pendência. Repetir em bancada com a placa.

S-16 — Firmware sem controle de versão

Problema. A placa não informava a versão; uma placa com firmware antigo, sem a autenticação de S-01, não seria percebida.

Correção (commit `290e028`). O firmware publica a versão depois de autenticar (`tennis_imu.ino` , linha 231). O app recusa versão anterior a 1.1.0. A API recusa iniciar sessão com raquete abaixo de 1.1.0 ou abaixo da versão exigida pelo treino (`sessoes.service.ts` , linhas 370 a 377).

Verificação. Cenário do firmware "prova correta autentica e publica a versão do firmware" e teste E2E "recusa sessão em raquete sem autenticação BLE (firmware < 1.1.0)".

Residual. A atualização é só por cabo USB e o firmware não é assinado: quem tiver acesso físico pode gravar outro programa.

S-17 — Acesso a dados de outra conta pela API (novo)

Problema. A API guarda contas, praticantes (inclusive menores), sessões e raquetes. Uma rota que não conferisse o dono permitiria ler ou alterar dados de outra pessoa trocando um ID (OWASP A01).

Controles (commit `5f88e47`). Toda rota exige login, salvo as marcadas como públicas; o guard é global (`app.module.ts` , linha 65). Rotas de autor e administrador conferem o perfil (`autenticacao.guard.ts` , linha 26). Cada recurso confere o dono e responde 404 quando não é dele, sem revelar que o ID existe:

```
async garantirPosse(praticanteId: string, contaId: string) {        //
praticantes.service.ts, linhas 25 a 28
  const p = await this.prisma.praticante.findUnique({ where: { id:
praticanteId } });
  if (!p || p.contaId !== contaId) throw naoEncontrado('Praticante');
```

Verificação. Testes E2E "jogador não cria treino" (403), "site lista os treinos do autor com rascunhos; admin vê todos" (jogador recebe 403), "entrega a chave da raquete só ao dono e guarda cifrada" e "dependente só executa treinos infantis ou para todos". No commit `48b5726`, três testes reproduzem o ataque: ler as sessões, alterar o nível ou ver os indicadores do praticante de outra conta responde 404, e o dado não muda; jogador recebe 403 nas rotas de administração; e rota protegida sem token responde 401.

S-18 — Ataques ao login da API (novo)

Problema. Login e cadastro públicos ficam expostos a tentativa de senhas, descoberta de e-mails cadastrados e roubo de sessão (OWASP A07).

Controles (commit `5f88e47`):

- Limite de 5 tentativas por minuto em login e cadastro (`auth.controller.ts`, linha 10), contado por IP e e-mail (`limite.guard.ts`, linhas 7 a 9).
- Senhas com bcrypt de custo 12 (`auth.service.ts`, linha 11); senha de 8 a 72 caracteres, com letras e números (`auth/dto.ts`).
- A mesma mensagem para e-mail inexistente e senha errada (linha 58), e o bcrypt roda mesmo quando o e-mail não existe, para o tempo de resposta não denunciar a conta (linhas 48 e 69).
- Token de acesso de 15 minutos e refresh de 7 dias, trocado a cada uso. Reusar um refresh já trocado revoga todos os tokens daquela sessão e registra um incidente (`tokens.service.ts`, linhas 49 a 58).

Verificação. Teste E2E "reuso de refresh token revoga a família" e, no commit `48b5726`, cinco testes que reproduzem ataques: token assinado com outro segredo (401, inclusive tentando se passar por administrador), token sem assinatura com `alg: none` (401), token expirado com o segredo certo (401), mesma resposta para e-mail inexistente e senha errada, e a sexta tentativa de login no minuto bloqueada com 429 mesmo com a senha certa.

Residual. Um token de acesso já emitido continua válido por até 15 minutos depois de a conta ser desativada; o refresh é revogado na hora.

S-19 — Privilégio excessivo no banco de dados (novo)

Problema. Se a API usasse o usuário administrador do banco, uma falha nela daria acesso a tudo, inclusive para apagar tabelas e a trilha de auditoria.

Controles (commit `5f88e47`, `prisma/sql/papeis.sql` e migration inicial):

- Dois papéis: `tennis_owner`, só para rodar migrations, e `tennis_api`, usado pela API em execução, só com leitura e escrita de dados.
- As tabelas ficam num schema próprio, `tennis`, e não no `public` que a API de dados do Supabase expõe. O acesso público ao schema é revogado (`papeis.sql`, linha 15).
- RLS ligado em todas as tabelas, com política só para `tennis_api` (migration inicial, linha 364).
- `tennis_api` não altera nem apaga a auditoria (linha 359).
- Conexão com `sslmode=require`; as senhas dos papéis ficam só no `.env` e no painel do deploy.

Verificação. Depois da troca de senhas, as antigas foram recusadas pelo banco e as novas aceitas. Os testes E2E rodam com o papel `tennis_api`, num schema separado (`tennis_test`).

S-20 — Chaves das raquetes guardadas no servidor (novo)

Problema. Para cada raquete ter a própria chave (S-01), a API precisa guardá-las. Um vazamento do banco entregaria a chave de todas as raquetes.

Controles (commit `290e028`):

- As chaves ficam cifradas com AES-256-GCM (`cifra.ts`, linha 3), com uma chave mestra que só existe no ambiente (`configuracao.ts`, linha 32).
- A coluna cifrada nunca sai do banco sem pedido explícito (`prisma.service.ts`, linha 5).
- Só o dono recebe a chave, numa rota sem cache (`dispositivos.controller.ts`, linha 25); cada entrega fica na auditoria (`dispositivos.service.ts`, linha 71).
- A chave pública de exemplo é recusada no cadastro da raquete (linha 35).

Verificação. 5 testes de `cifra.spec.ts`, entre eles "recusa texto adulterado (GCM autentica)" e "recusa decifrar com outra chave mestra", e o teste E2E "entrega a chave da raquete só ao dono e guarda cifrada".

Residual. O app guarda uma cópia da chave no SecureStore para parear sem internet. Trocar a chave mestra exige recadastrar as raquetes.

S-21 — Configuração HTTP insegura na API (novo)

Problema. Cabeçalhos ausentes, CORS aberto, campos inesperados aceitos, mensagens de erro com detalhes internos ou respostas em cache expõem a API (OWASP A05).

Controles (`app.setup.ts` e `configuracao.ts`):

- Cabeçalhos de segurança do Helmet em todas as rotas; só a documentação relaxa a CSP (linhas 50 a 52).
- CORS só para as origens configuradas (linha 61).
- Corpo validado em todas as rotas, recusando campos não previstos (linhas 23 e 24).
- Erros inesperados não expõem stack trace nem mensagem interna (`erro.filter.ts`).
- `Cache-Control: no-store` em toda resposta, para dados pessoais não ficarem no cache do navegador (linhas 54 a 58, commit `1050afe`).
- Os logs mascaram o cabeçalho Authorization e os cookies (`app.module.ts`, linha 40).
- O endereço da plataforma mãe vem só do ambiente, nunca de dado do usuário (`configuracao.ts`, linha 34), o que evita SSRF (OWASP A10).

Correções do commit `48b5726`. Corpo acima de 100 KB passou a responder 413 no formato da API, sem detalhes internos; antes respondia 500 (`erro.filter.ts`). E a API não sobe se um segredo obrigatório ainda tiver o valor de exemplo do `.env.example`, que é público (`configuracao.ts`).

Verificação. Teste E2E "respostas com dados pessoais não ficam no cache do navegador" e, no commit `48b5726`, seis testes que reproduzem ataques: cabeçalhos do Helmet presentes e sem `X-Powered-By`; CORS libera a origem do site e não libera uma origem desconhecida; campo não previsto no corpo recusado com 400; JSON malformado recusado com 400, sem detalhes internos; corpo de 200 KB recusado com 413; e ID fora do formato UUID recusado antes de chegar ao banco. Cinco testes de unidade (`configuracao.spec.ts`) conferem a recusa de segredos curtos ou de exemplo.

Varredura com OWASP ZAP (29/10/2026). ZAP 2.17.0 no modo de automação, contra a API rodando no computador em modo de produção, ligada ao schema `tennis_test`. As 56 URLs vieram da especificação OpenAPI da própria API. A varredura ativa usou o token de uma conta de jogador criada para o teste, uma requisição por vez, e parou no limite de 25 minutos que definimos, antes de percorrer todas as regras. A API respondeu a 2.508 requisições, nenhuma com erro 500; a maioria foi recusa: 1.362 com 403, 966 com 400, 98 com 429 e 32 com 401.

Resultado do ZAP. Um alerta: Path Traversal, risco alto e confiança baixa, no campo `apelido` de `POST /api/v1/praticantes/dependentes`, com o valor de ataque "dependentes" e sem evidência na resposta. Analisamos como falso positivo: a API não lê nem grava arquivo com dado do usuário (só `carregar-env.ts` e `exportar-openapi.ts`

usam o sistema de arquivos, sem entrada externa), e `apelido` é um texto de 1 a 30 caracteres gravado no banco. O valor enviado é aceito como um apelido qualquer. O relatório do ZAP está no repositório, na pasta Documentos/Entrega02/Redes de Computadores e Cibersegurança.

Pendência. Repetir a varredura na API implantada, com HTTPS (S-24), sem limite de tempo e também com uma conta de administrador: com a conta de jogador, as rotas de administração responderam 403 e não foram exercitadas por dentro.

S-22 — Roubo de sessão e clickjacking no site (novo)

Problema. Um script injetado no site poderia ler os tokens do usuário, e outro site poderia exibir o nosso dentro de um frame para induzir cliques.

Controles (commit `e202385`):

- O token de acesso fica só em memória. O refresh fica no `sessionStorage`: some ao fechar a aba e não passa para outras abas (`Sessao.tsx`, linhas 7 a 22).
- O deploy define uma CSP sem script inline nem de terceiros, com `connect-src` só para a API e `frame-ancestors 'none'`, além de `nosniff`, `Referrer-Policy: no-referrer` e HSTS (`render.yaml`). O `vite preview` aplica a mesma CSP.
- As fontes são servidas pelo próprio site, sem Google Fonts, para o IP de quem acessa não ir a terceiros.

Verificação. Em 25/10/2026, o build de produção foi servido com a CSP e percorrido num navegador automatizado: login, criação, validação, ensaio, revisão, publicação e marketplace, sem nenhuma violação de CSP e sem erro de JavaScript.

Residual. Enquanto a página estiver aberta, um script que furasse a CSP leria o refresh do `sessionStorage`.

S-23 — Dependências da API e do site (novo)

Problema. Bibliotecas com falhas conhecidas na API e no site (OWASP A06).

Situação em 28/10/2026. O site tinha 0 vulnerabilidades. A API tinha 2 altas em produção, ambas em `multer` 2.4.0, puxado por `@nestjs/platform-express` 11.2.7 (negação de serviço no envio de arquivos), embora a API não tenha rota de envio de arquivo. O commit `5f88e47` já tinha forçado `deepmerge-ts` e `js-yaml` para versões corrigidas.

Correção (commit `48b5726`). `multer` forçado para 2.5.0, publicado em 22/10/2026 e fora da faixa vulnerável (2.0.0 a 2.4.0). Em 29/10/2026, `npm audit --omit=dev` da API: 0 vulnerabilidades. Contando as ferramentas de desenvolvimento, restam 9 baixas em `deepmerge`, usado pelo Jest e pelo Nest CLI, que não vão para o servidor. O passo de auditoria do workflow da API passou no GitHub, na main (execução 40812441168).

S-24 — Tráfego sem TLS fora do ambiente local (novo)

Problema. Sem HTTPS, senhas e tokens trafegariam legíveis entre o app ou o site e a API.

Resposta (Transferir). No deploy, o Render termina o TLS da API e do site, e o site manda HSTS. A conexão da API com o banco já exige TLS (`sslmode=require`).

Pendência. Fazer o deploy e repetir a varredura com OWASP ZAP na API implantada (S-21). Hoje, o endereço da API no app vem de `EXPO_PUBLIC_API_URL` , e nada no código impede um endereço `http://` num build de produção.

Anexo B — Verificação

Em 28/10/2026, todos os testes passaram sobre o commit `8431cad` : 27 do firmware, 74 do app, 114 de unidade e 17 E2E da API, e 19 do site. As execuções foram feitas numa cópia limpa da branch, com dependências instaladas do zero (`npm ci`). Em 29/10/2026, sobre a `main` (commit `e8a2fec`), a API passou a ter 119 testes de unidade e 34 E2E, todos aprovados.

Execuções de 29/10/2026

Componente	Comando	Resultado
API	`npx jest` (unidade)	119 testes, 119 aprovados
API	`npm run test:e2e` (contra o schema `tennis_test`)	34 testes, 34 aprovados
API	`npm audit --omit=dev`	0 vulnerabilidades
API	`npm audit` (com ferramentas de desenvolvimento)	9 baixas (deepmerge)
API	OWASP ZAP 2.17.0, varredura local (Anexo A, S-21)	1 alerta, analisado como falso positivo
GitHub	`backend.yml` na `main` (execução 40812441168)	sucesso
GitHub	`seguranca.yml` na `main` (execução 40812441260), com gitleaks em todo o histórico	sucesso, nenhum vazamento

Execuções de 28/10/2026

Componente	Comando	Resultado
Firmware	`zig c++ -std=c++17 -Wall -Wextra -Werror teste_autenticacao.cpp` e execução	27 testes, 0 falhas
App	`npm test`	74 testes, 74 aprovados
App	`npx tsc --noEmit`	sem erros
App	`npm audit`	13 vulnerabilidades: 2 altas, 11 baixas
API	`npx tsc --noEmit`	sem erros
API	`npx jest` (unidade)	114 testes, 114 aprovados
API	`npm run test:e2e` (contra o schema `tennis_test`)	17 testes, 17 aprovados
API	`npm audit --omit=dev`	2 altas (`multer`)
API	`npm audit` (com ferramentas de desenvolvimento)	11: 2 altas, 9 baixas
Site	`npm test`	19 testes, 19 aprovados
Site	`npm run build` (tipos + build)	sem erros
Site	`npm audit`	0 vulnerabilidades
Repositório	`gitleaks detect` em todo o histórico da branch	27 commits analisados, nenhum vazamento

Execuções de 25/10/2026

- Compilação do firmware para a UNO R4 WiFi com o `arduino-cli`: 40% da memória flash e 37% da RAM.
- App no SDK 54: `expo-doctor` com 18 de 18 verificações, pacote Android gerado com `expo export` e `expo prebuild` conferindo as permissões do `AndroidManifest.xml`.
- Site: fluxo completo num navegador automatizado sobre o build de produção com a CSP, contra a API local ligada ao schema `tennis_test`, sem violação de CSP nem erro de JavaScript.

Testes ligados a cada risco

Risco	Testes
S-01, S-03, S-09, S-15, S-16	27 cenários do firmware (`teste_pc/teste_autenticacao.cpp`); `autenticacao.test.ts` e `hmac.test.ts` no app
S-12, S-13	`protocolo.test.ts` no app
S-04, S-17	E2E: promoção a autor, jogador sem acesso de autor, lista de autoria, dependente restrito; praticante de outra conta (404), rotas de administração (403) e rota sem token (401)
S-08	E2E: emergência e comando bloqueado auditados como incidente
S-13	E2E: lote duplicado e sincronização offline idempotente
S-15	`estado.spec.ts` e E2E da sessão com parada segura
S-16	E2E: sessão recusada com firmware anterior a 1.1.0
S-18	E2E: reuso de refresh token; token de outro segredo, sem assinatura (`alg: none`) e expirado; mesma resposta para e-mail inexistente; sexta tentativa de login bloqueada (429)
S-20	`cifra.spec.ts` (5 testes) e E2E da chave da raquete
S-21	E2E: `Cache-Control: no-store`, cabeçalhos, CORS, campo não previsto, JSON malformado, corpo grande (413) e ID inválido; `configuracao.spec.ts` (5 testes); varredura local com OWASP ZAP
S-07	E2E: exportação dos dados, exclusão de conta e retenção

No GitHub, os workflows `backend.yml` e `seguranca.yml` rodam a cada push os testes do firmware, do app e de unidade da API, a verificação de tipos, a compilação do firmware com a chave de exemplo, o gitleaks e o `npm audit`. Em 29/10/2026, os dois passaram na `main` (execuções 40812441168 e 40812441260). Os testes E2E da API só rodam no GitHub depois que o grupo configurar a variável e os segredos do schema de teste.

Anexo C — Mapa das alterações no código

Dez commits, de 23/10 a 29/10/2026, levam o repositório do commit `74ee92e` (22/10/2026) ao `48b5726`. Os dois primeiros estão na branch `feat/backend`; a `feat/seguranca` parte dela e tem os outros oito. A branch foi enviada ao GitHub em 29/10/2026 e mesclada na `main` pelo commit `e8a2fec`.

Commit	Data	Arquivos	Linhas (+/−)	O que fez	Riscos
`48b5726`	29/10/2026	7	+315 / −5	17 testes E2E que reproduzem ataques; corpo grande demais passa a responder 413; API recusa segredos de exemplo; multer 2.5.0	S-07, S-17, S-18, S-21, S-23
`8431cad`	25/10/2026	20	+3.986 / −9.309	Expo SDK 54, React Native 0.81, React Navigation 7	S-05
`e202385`	25/10/2026	33	+5.407	Site de autoria e administração, CSP e cabeçalhos no deploy, job do site no CI	S-22, S-23
`1050afe`	25/10/2026	7	+86 / −5	`Cache-Control: no-store` na API e lista de treinos do autor	S-21
`dadcbac`	25/10/2026	48	+3.515 / −1.144	App ligado à API: login, sessão, parada segura, perfil, exportação e exclusão	S-04, S-06, S-07, S-13, S-15
`290e028`	25/10/2026	33	+1.267 / −29	Desafio e resposta no BLE, versão do firmware, chaves cifradas na API	S-01, S-02, S-03, S-09, S-12, S-16, S-20
`2dea3b9`	25/10/2026	10	+128 / −23	Validação da duração dos treinos (sem impacto de segurança)	—
`38d8bf9`	25/10/2026	10	+68 / −23	Dado pessoal fora do app, depuração só em desenvolvimento, `.expo` fora do Git, workflow `seguranca.yml`	S-06, S-09, S-10, S-11
`21abeb5`	23/10/2026	2	+15 / −1	README e configuração do MCP do Supabase	—
`5f88e47`	23/10/2026	95	+19.499	API NestJS, banco, auditoria, privacidade, deploy e CI da API	S-04, S-07, S-08, S-11, S-13, S-15, S-17, S-18, S-19, S-21, S-23

Nas linhas de `8431cad`, a maior parte vem do `package-lock.json` do app, regenerado na troca do SDK.

Anexo D — Limitações e próximos passos

O que ainda não foi comprovado depende de hardware, de aparelho ou de publicação, e não de código novo:

- **Placa:** nenhum cenário do firmware foi repetido com a UNO R4 WiFi e o celular; os 27 cenários rodam no PC.
- **Aparelho:** o app no SDK 54 não foi instalado num celular; o pacote foi gerado, mas o BLE não foi testado nessa versão.
- **Publicação:** a API e o site não foram implantados. Sem isso não há HTTPS de produção (S-24), e a varredura com OWASP ZAP foi feita só na API local, com conta de jogador e limite de 25 minutos (S-21).
- **GitHub:** os testes E2E da API ainda não rodam no GitHub; dependem da variável e dos segredos do schema de teste. Hoje eles rodam no computador de quem desenvolve.
- **Histórico do Git:** o e-mail pessoal (S-06) e a pasta `.expo` (S-10) continuam nos commits anteriores à correção.
- **`npm audit`:** reflete as vulnerabilidades conhecidas na data da execução e não separa o que vai para o aparelho do que é só ferramenta de build.

Próximos passos

- [x] ~~Push da branch `feat/seguranca` e primeira execução dos workflows no GitHub (S-11)~~
- [x] ~~Forçar `multer` 2.5.0 na API e repetir os testes (S-23)~~
- [x] ~~Testes automáticos do limite de tentativas do login (S-18), de CORS e cabeçalhos (S-21) e de exportação, exclusão e retenção (S-07)~~
- [ ] Build novo do app e teste no celular, com atenção ao BLE (S-05)
- [ ] Teste em bancada: autenticação, parada segura e versão do firmware (S-01, S-03, S-15, S-16)
- [ ] Verificar o pareamento com criptografia da ArduinoBLE na UNO R4 WiFi (S-02)
- [ ] Deploy da API e do site no Render e nova varredura com OWASP ZAP na API implantada, sem limite de tempo e também com conta de administrador (S-21, S-24)
- [ ] Confirmar em reunião a classificação de S-17 a S-24 feita na seção 2 e os níveis residuais propostos no Anexo E

Anexo E — Nível residual (proposta)

Proposta para o grupo confirmar em reunião. Usa a mesma matriz de probabilidade × impacto da seção 7 do Plano da Entrega 1. S-01 a S-16 repetem o nível residual da análise do grupo de 24/10/2026; S-17 a S-24 foram classificados agora. Para S-02, S-05 e S-24, ainda

Em Andamento, o valor é o esperado ao concluir. As respostas reduzem a probabilidade; o impacto fica igual, porque o dano de um ataque bem-sucedido não muda.

Risco	Antes	Depois (residual)	Por quê
S-01	Alto (Média × Alto)	Baixo (Muito Baixa × Alto)	Desafio e resposta com chave por raquete; falta repetir em bancada
S-02	Médio (Média × Baixo)	Baixo (Baixa × Baixo), esperado	Nenhum segredo passa pelo canal; falta verificar o pareamento com criptografia
S-03	Baixo (Baixa × Médio)	Baixo (Muito Baixa × Médio)	A placa prova primeiro que conhece a chave
S-04	Alto (Alta × Médio)	Baixo (Baixa × Médio)	Login pela API e perfis conferidos no servidor
S-05	Alto (Média × Alto)	Médio (Baixa × Alto), esperado	SDK 54; restam 2 altas em ferramentas de build; falta o teste no celular
S-06	Médio (Alta × Baixo)	Baixo (Baixa × Baixo)	Dado pessoal fora do código; continua no histórico do Git
S-07	Alto (Alta × Médio)	Baixo (Baixa × Médio)	Direitos do titular implementados e testados; a idade é declarada
S-08	Médio (Alta × Baixo)	Baixo (Baixa × Baixo)	Trilha de auditoria que a API não altera nem apaga
S-09	Médio (Média × Baixo)	Muito Baixo (Muito Baixa × Baixo)	Tela só em build de desenvolvimento
S-10	Baixo (Alta × Muito Baixo)	Muito Baixo (Muito Baixa × Muito Baixo)	.expo fora do controle de versão
S-11	Médio (Baixa × Muito Alto)	Médio (Muito Baixa × Muito Alto)	Segredos fora do Git e gitleaks a cada push; o impacto de um vazamento continua Muito Alto
S-12	Baixo (Baixa × Médio)	Baixo (Baixa × Médio)	Controles já existiam; desafio e prova só em hexadecimal
S-13	Baixo (Baixa × Médio)	Baixo (Baixa × Médio)	Sequência no app e UUID por evento na API

Risco	Antes	Depois (residual)	Por quê
S-14	Muito Baixo (Muito Baixa × Baixo)	Muito Baixo (Muito Baixa × Baixo)	Sem localização no Android 12 ou superior
S-15	Médio (Baixa × Alto)	Baixo (Muito Baixa × Alto)	Parada segura no firmware, no app e na API; falta repetir em bancada
S-16	Médio (Média × Médio)	Baixo (Baixa × Médio)	Versão mínima 1.1.0 conferida no app e na API
S-17	Alto (Média × Muito Alto)	Médio (Baixa × Muito Alto)	Dono conferido em cada recurso e testado; uma rota nova sem a checagem ainda exporia dados, inclusive de menores
S-18	Alto (Média × Alto)	Médio (Baixa × Alto)	Limite de tentativas, bcrypt e tokens testados; token já emitido vale até 15 minutos
S-19	Alto (Média × Muito Alto)	Médio (Muito Baixa × Muito Alto)	Papel só com leitura e escrita, schema próprio e RLS
S-20	Médio (Baixa × Alto)	Baixo (Muito Baixa × Alto)	Chaves cifradas com AES-256-GCM e chave mestra só no ambiente
S-21	Médio (Média × Médio)	Baixo (Baixa × Médio)	Configuração testada e ZAP local sem achado real
S-22	Médio (Baixa × Alto)	Baixo (Muito Baixa × Alto)	CSP sem script de terceiros e site fora de frames
S-23	Alto (Média × Alto)	Médio (Baixa × Alto)	Nenhuma vulnerabilidade em produção hoje; novas podem surgir, e o `npm audit` roda a cada push
S-24	Alto (Média × Alto)	Alto até o deploy; esperado Baixo (Muito Baixa × Alto)	TLS do Render e HSTS no site

Resumo. Antes: 9 Altos, 10 Médios, 4 Baixos e 1 Muito Baixo. Depois: 1 Alto (S-24, até o deploy), 6 Médios, 14 Baixos e 3 Muito Baixos. Os seis Médios (S-05, S-11, S-17, S-18, S-19 e S-23) têm impacto Alto ou Muito Alto: mesmo com probabilidade baixa, a matriz os mantém em Médio, e por isso seguem no monitoramento semanal (seção 4).